Android 低内存设备内存排查指南

版本号: 2.0

发布日期: 2025.09.18

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
2.0	2025.09.18	AWA0681	初始版本

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 概述 1

1.1 编写目的 1

1.2 适用范围 1

1.3 相关人员 1

1.4 内存压力测试 1

1.4.1 测试内存读写 1

1.4.1.1 测试环境的搭建 2

1.4.2 其他内存压力测试方法 2

1.5 内存分析工具 2

1.5.1 procrank 2

1.5.2 cat proc/meminfo 3

1.5.3 dumpsys meminfo 4

1.5.4 ramparser 6

1.5.5 slabinfo 7

1.5.6 dmabuf_dump 7

1.5.7 cpu_monitor 8

1.6 内存相关的常用调试节点 9

1.6.1 proc/meminfo 9

1.6.2 proc/vmallocinfo 10

1.6.3 proc/buddyinfo 10

1.6.4 proc/pagetypeinfo 10

1.6.5 proc/zoneinfo 10

1.6.6 proc/vmstat 11

2 Android 上的内存管理机制 12

2.1 lmkd 12

2.1.1 lmkd 的配置 12

2.1.1.1 调整 ro.lmk.downgrade_pressure 12

2.1.1.2 调整 ro.lmk.upgrade_pressure 12

2.1.1.3 调整 ro.lmk.critical 13

2.2 kswapd 13

2.2.1 内核的内存回收机制 13

2.2.2 内存水位标记 13

2.2.3 触发 kswapd 的时机 13

2.2.4 触发 kswapd 对系统的影响 14

2.3 ZRAM 14

2.3.1 打开 zram 的配置 14

2.3.2 设置 zram 的参数 15

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.3.2.1 zram 的大小 15
2.3.2.2 zram 的压缩算法 15
2.3.2.3 zram 的最大压缩流 15
2.3.3 ZRAM 回写配置 15

3 常用的内存优化方法 17
3.1 设置为 low device 17
3.2 优化预留内存 17
3.2.1 如何查看详细的预留内存 17
3.3 优化 GC 回收机制 19
3.4 优化 awbms 19
3.5 优化 SystemUI 19
3.5.1 裁剪法优化 SystemUi 19
3.6 通过虚拟机参数优化 SystemServer 20
3.7 gms 包的裁剪 20
3.8 优化 vmalloc 20
3.9 优化 slab 20
3.10 裁剪不必要的应用 20
3.11 关掉蓝牙和自动旋转 20
3.12 显示优化 21
3.13 合理配置 cma 的预留大小 21

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

插　图

图 1-1　cpu_monitor 9
图 2-1　lmkd 配置 12
图 2-2　lz4 15
图 2-3　zram 最大压缩流 15
图 3-1　memoryblock_reserve 17
图 3-2　memoryblock_reserve2 17
图 3-3　systemui 优化 19
图 3-4　SurfaceFlinger 21

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 概述

1.1 编写目的

- 本文档提供 Android 低内存设备内存优化指南
- 本文档提供的快速排查指南工程思路如下：内存调式工具 + 内存常见参数 + Android 内存管理 + 常见优化措施
- 本文档仅用于帮助开发人员快速排查和解决初级问题问题，如果按照文档中的指导仍无法解决问题，请联系全志原厂 FAE 进一步协助解决，并提供说明是否按照文档中的排查方案进行过必要的排查，提供排查过程的必要记录和数据

1.2 适用范围

本文档基于全志平台编写，适用于 Android Go 设备，需要区分 GMS 设备和非 GMS 设备。

1.3 相关人员

Android 系统工程师，性能优化工程师，内存优化工程师。

1.4 内存压力测试

新的方案必须开展内存压力测试，内存压力测试有两个作用，第一个作用是保证硬件 DRAM 的读写是稳定的，这样才能保证系统不会出现异常跑飞问题；第二个作用是在保证第一个的前提下，测试系统的内存使用，以此来证明系统的内存管理是没问题的，系统是不存在内存泄漏的，系统的内存是足够够用的。

1.4.1 测试内存读写

为保证 DRAM 不影响系统的稳定性，必须先做测试。测试场景是 memtetser+ 捕鱼达人 app。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.4.1.1 测试环境的搭建

1. 安装捕鱼达人 app，运行
2. 串口控制台执行 memteser 128m 1000 0 &
3. 如果测试 48 小时系统不会出现死机、退出等问题，则认为当前系统的 DRAM 是稳定的。

针对不同内存设备，可以设置不同的测试参数，对于 1G 设备设置 256m，2G 设备设置 512m，其余自行参考设定，不能总体上不要超过系统总内存的 25%

1.4.2 其他内存压力测试方法

除了捕鱼 memtester 内存测试方法，还可以使用 monkeytest 进行内存压力测试，可以在 KeepTesting 中启动 monkey 测试，测试验收标准为 48h 内没有出现硬件导致的卡死问题

1.5 内存分析工具

全志平台下常用的内存分析工具：

```
procrank
cat /proc/meminfo
dumpsys meminfo
ramparser
slabinfo
dmabuf_dump
cpu_monitor
```

1.5.1 procrank

此命令可以直观获知当前系统的可用内存情况

```
demo:/ # procrank
  PID       Vss      Rss      Pss      Uss     Swap    PSwap    USwap    ZSwap  cmdline
  286    11556K    2584K       4K       4K     672K     672K     672K      78K  /vendor/bin/
    hw/android.hardware.cas-service.example
  173    15020K    2716K       4K       4K     936K     936K     936K     109K  /vendor/bin/
    hw/android.hardware.security.keymint@3.0-service-aw
  264    12880K    2780K       4K       4K     752K     752K     752K      88K  /vendor/bin/
    hw/android.hardware.sensors@2.0-service
  406    54448K    3936K       4K       4K    3492K    3492K    3492K     408K  /system/bin/
    systemmixservice
                          ------   ------   ------   ------   ------   ------   ------
                         151242K  134616K  288200K  157929K  133220K   18462K  TOTAL

ZRAM: 22540K physical used for 169472K in swap (427612K total swap)
 RAM: 475132K total, 61944K free, 2476K buffers, 126872K cached, 1248K shmem, 73424K slab 184
    M free+cached, 395M used
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

其中的 ZRAM 表示当前系统开启的 ZRAM 大小，如果在 ZRAM 项目为 0，需要排查 fstab 中 ZRAM 部分的配置有没有异常。

RAM 部分表示软件能够使用的内存，上面的用例总 DDR 大小 512M，但内核和 uboot、secure drm 等模块会预留一部分物理内存，导致可使用的内存小于 512M，因此显示可用内存为 470M 左右，如果这个数值太小，需要检查 secure 的内存是否有配置正确，free+cached 项为当前系统的第三方软件真正能够使用的内存数量，也是内存优化的判断依据。

1.5.2 cat proc/meminfo

此命令通过内核节点获知内存的占用情况，可以通过这个命令获知 slab/vmalloc 等内存的占用情况，下面为一个范例：

```
demo:/ # cat /proc/meminfo
MemTotal:         1006468 kB      //系统总共可以使用的内存总大小
MemFree:          55340 kB       //当前系统剩余内存大小
MemAvailable:      526196 kB      //可回收内存+memfree的内存
Buffers:             1956 kB       //用于文件缓冲的内存大小
Cached:            524704 kB        //用于高速缓存的内存大小

SwapCached:           788 kB      //用于swap的缓存

Active:            392392 kB       //活跃使用状态，记录最近使用的内存，不可回收。
Inactive:           334508 kB       //非活跃使用状态，记录最新可以被使用的内存，可回收。
Active(anon):       113848 kB       // Active = Active(anon) + Active(file)
Inactive(anon):      114740 kB       // Inactive = Inactive(anon) + Inactive(file)
Active(file):         278544 kB
Inactive(file):        219768 kB

Unevictable:        27564 kB
Mlocked:           25256 kB

HighTotal:          262144 kB       //高端内存总大小，256M（ARMv7 32bit才有高端地址的概念）
HighFree:             5100 kB       //高端内存空闲内存大小
LowTotal:          744324 kB       //低端内存总大小，726M
LowFree:            50240 kB       //低端内存空闲内存大小

SwapTotal:         754844 kB       //swap总大小
SwapFree:          749728 kB      //swap可用大小
Dirty:                 24 kB
Writeback:              0 kB

AnonPages:         227608 kB      //匿名页，用户空间的页表
Mapped:           295856 kB      //文件通过mmap分配的内存，用于map设备、文件或者库
Shmem:            1052 kB

Slab:              57824 kB       //slab的总大小
SReclaimable:       29332 kB       //slab中可回收的大小
SUnreclaim:         28492 kB       //slab中不可回收的大小

KernelStack:         6768 kB
PageTables:         14972 kB
NFS_Unstable:           0 kB
Bounce:                 0 kB
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
WritebackTmp:          0 kB
CommitLimit:     1258076 kB
Committed_AS:   17852900 kB

VmallocTotal:     245760 kB          //总分配的虚拟地址空间
VmallocUsed:           0 kB        //4.9内存这里统计有问题，可查看/proc/vmallocinf看实际使用情况
VmallocChunk:          0 kB

CmaTotal:           8192 kB        //预留cma总大小
CmaFree:            1584 kB        //cma剩余可用内存
```

1.5.3 dumpsys meminfo

dumpsys meminfo 是 Android 上最重要的内存调试命令，其列出的项目意义如下：

类型	解释
process	以进程的 PSS 从大到小依次排序显示，每行显示一个进程
OOM adj	Native/System/Persistent/…./BServices/Cached，分别显示每类的进程情况
category	以 Dalvik/Native/.art mmap/.dex map 等划分各类进程的总 PSS 情况
total	总内存、剩余内存、可用内存、Lost 内存、ZRAM 使用情况

下面的信息为一个范例 (为了减少篇幅信息做了删减)

```
demo:/ # dumpsys meminfo
Applications Memory Usage (in Kilobytes):
Uptime: 2277186 Realtime: 2277186


Total RSS by process:
    108,244K: system (pid 464)
     41,828K: com.softwinner.firelauncher (pid 838 / activities)
     36,328K: android.process.media (pid 1135)
     33,932K: com.android.providers.media.module (pid 983)
     33,752K: com.android.networkstack.process (pid 793)
     24,416K: com.softwinner.awtvinputservice (pid 776)
     18,832K: zygote (pid 243)
     ......
Total RSS by OOM adjustment:
    239,872K: Native
         18,832K: zygote (pid 243)
         14,352K: webview_zygote (pid 764)
          8,500K: wpa_supplicant (pid 1637)
          7,868K: surfaceflinger (pid 295)
          5,160K: audioserver (pid 291)
    108,244K: System
         108,244K: system (pid 464)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
     33,752K: Persistent
         33,752K: com.android.networkstack.process (pid 793)
     33,932K: Persistent Service
         33,932K: com.android.providers.media.module (pid 983)
     41,828K: Foreground
         41,828K: com.softwinner.firelauncher (pid 838 / activities)
     24,416K: Visible
         24,416K: com.softwinner.awtvinputservice (pid 776)
     36,328K: Cached
         36,328K: android.process.media (pid 1135)
       ......
Total RSS by category:
    208,428K: .so mmap
     90,392K: .jar mmap
      ......

Total PSS by process:
     89,836K: system (pid 464)
     46,776K: com.softwinner.firelauncher (pid 838 / activities)
     15,355K: com.android.providers.media.module (pid 983)
     14,524K: com.android.networkstack.process (pid 793)
     13,649K: android.process.media (pid 1135)
     11,224K: surfaceflinger (pid 295)
     10,420K: webview_zygote (pid 764)
     10,138K: com.softwinner.awtvinputservice (pid 776)
      9,635K: zygote (pid 243)
      5,529K: wpa_supplicant (pid 1637)
      4,761K: mediaserver (pid 404)
      4,497K: audioserver (pid 291)
      ......

Total PSS by OOM adjustment:
    123,280K: Native
         11,224K: surfaceflinger (pid 295)
         10,420K: webview_zygote (pid 764)
          9,635K: zygote (pid 243)
          ......

     89,836K: System
         89,836K: system (pid 464)
     14,524K: Persistent
         14,524K: com.android.networkstack.process (pid 793)
     15,355K: Persistent Service
         15,355K: com.android.providers.media.module (pid 983)
     46,776K: Foreground
         46,776K: com.softwinner.firelauncher (pid 838 / activities)
     10,138K: Visible
         10,138K: com.softwinner.awtvinputservice (pid 776)
     13,649K: Cached
         13,649K: android.process.media (pid 1135)
          ......
Total PSS by category:
     41,891K: .jar mmap
     32,890K: .so mmap
     16,226K: Native
     16,142K: .art mmap
     15,950K: Dalvik
     ......
Total RAM:   475,132K (status normal)
 Free RAM:   124,677K (   13,649K cached pss +    56,152K cached kernel +    54,876K free)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
DMA-BUF:    66,568K (   65,932K mapped +       636K unmapped)
DMA-BUF Heaps:    66,568K
DMA-BUF Heaps pool:         0K
      GPU:          0K
 Used RAM:   439,789K (  365,841K used pss +    73,948K kernel)
 Lost RAM:    41,293K
     ZRAM:    22,212K physical used for   173,312K in swap (  427,612K total swap)
   Tuning: 128 (large 256), oom   184,320K, restore limit    61,440K (low-ram)
```

输入 dumpsys meminfo 可以获知系统内存情况，如果打印出现超时，可以使用 dumpsys -t 1000 meminfo 命令，这个命令可以获知单个进程占用的内存情况，在开启 ZRAM 之后，PSS 项目显示的进程占用内存是算上 ZRAM 的，因此使用此命令跟踪内存更加准确。

使用 dumpsys meminfo pid 可以进一步观察单个进程的内存占用情况。

DMA-BUF 项为系统显示、Camera、多媒体等硬件占用的物理内存大小，此部分如果增大，需要检查显示多媒体是否存在 buffer 泄露。

1.5.4 ramparser

使用 rampaser 命令可以观察内核或其他硬件占用内存的情况，范例如下：ramparser -k 命令可以打印出内核占用内存的情况

```
demo:/ # ramparser  -k

Kernel used       :  71.62 MB
Kernel reserved   :  48.00 MB
Kernel used total : 119.62 MB

Kernel used  = Slab_Unrecalim(44.68MB) + KernelStack(5.15MB) +
               PageTables(10.49MB) + Shmem(1.28MB) + Modules(10.02MB)
Modules      = vmalloc(5.40MB) + insmod_ko(4.62MB)

Mem Used Summary(kernel space):
Mem     Total = 512 MB   (Free 47.53 MB, Available 89.89 MB, Cached 122.14 MB)
Mem Used Total = 119.62 MB
               = Kernel used total(119.62MB)
```

ramparser -m 命令可以打印出显示和多媒体占用内存的情况：

```
demo:/ # ramparser -m

Media total = 65.01MB
            =  + unknown(65.01MB)

unattached: (8.34MB)
28KB*9 8288KB*1

gpu: (40.47MB)
8288KB*5 4KB*1

vs-osd + gpu: (15.82MB)
8100KB*2

audbrg@203042c: (0.38MB)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
384KB*1
```

这个命令可以用来分析是哪个硬件模块占用了大量的内存。

1.5.5 slabinfo

slabinfo 命令可以用来分析内核的 slab 内存消耗情况，可以用于排查异常的驱动内存泄露等问题, 重点观察 kmalloc 部分的内置块分配情况：

```
demo:/ # slabinfo
Name                   Objects Objsize           Space Slabs/Part/Cpu  O/S O %Fr %Ef Flg
aio_kiocb                   64     128            8.1K          0/0/2   32 0   0 100 A
anon_vma                 14788      88            1.5M      335/35/34   42 0   9  86
anon_vma_chain           26175      32         897.0K      160/31/59  128 0  14  93

.........
kmalloc-128               8314     128            1.0M       243/8/18   32 0   3  99
kmalloc-192               2562     192          499.7K        87/0/35   21 0   0  98
kmalloc-1k                1343    1024            1.4M        70/8/16   16 2   9  97
kmalloc-256                672     256          172.0K        19/0/23   16 0   0 100
kmalloc-2k                 688    2048            1.4M         35/0/8   16 3   0 100
kmalloc-4k                 960    4096            3.9M       109/0/11    8 3   0 100
kmalloc-512               1971     512            1.0M        96/4/28   16 1   3  99
kmalloc-64               40103      64            2.6M      584/64/54   64 0  10  98
kmalloc-8k                  96    8192          786.4K        14/0/10    4 3   0 100

........
```

1.5.6 dmabuf_dump

dmabuf_dump 命令可以用于分析 Android 显示和多媒体分配的内存块，可以排查这部分内存的泄露：

```
demo:/ # dmabuf_dump
 er.firelauncher:838
                Name            Rss            Pss       nr_procs          Inode
      system-uncached       8288 kB        2762 kB              3             85
      system-uncached         28 kB           9 kB              3             86
      system-uncached       8288 kB        2762 kB              3             87
      system-uncached         28 kB           9 kB              3             88
      system-uncached       8288 kB        2762 kB              3             89
      system-uncached         28 kB           9 kB              3             90
      system-uncached       8288 kB        4144 kB              2             91
      system-uncached         28 kB          14 kB              2             92
      system-uncached       8288 kB        2762 kB              3             93
      system-uncached         28 kB           9 kB              3             94
      system-uncached       8288 kB        8288 kB              1             95
      system-uncached         28 kB          28 kB              1             96
        PROCESS TOTAL      49896 kB       23561 kB
---------------------
  surfaceflinger:295
                Name            Rss            Pss       nr_procs          Inode
      system-uncached          4 kB           4 kB              1              2
      system-uncached         28 kB          28 kB              1              3
```

```
                 system          8100 kB          4050 kB                2                4
       system-uncached            28 kB            14 kB                2                5
                 system          8100 kB          4050 kB                2                6
       system-uncached            28 kB            14 kB                2                7
       system-uncached          8288 kB          2762 kB                3               85
       system-uncached            28 kB             9 kB                3               86
       system-uncached          8288 kB          2762 kB                3               87
       system-uncached            28 kB             9 kB                3               88
       system-uncached          8288 kB          2762 kB                3               89
       system-uncached            28 kB             9 kB                3               90
       system-uncached          8288 kB          4144 kB                2               91
       system-uncached            28 kB            14 kB                2               92
       system-uncached          8288 kB          2762 kB                3               93
       system-uncached            28 kB             9 kB                3               94
         PROCESS TOTAL         57868 kB         23405 kB
----------------------
 composer@2.2-se:255
                 Name               Rss               Pss          nr_procs            Inode
                 system          8100 kB          4050 kB                2                4
       system-uncached            28 kB            14 kB                2                5
                 system          8100 kB          4050 kB                2                6
       system-uncached            28 kB            14 kB                2                7
       system-uncached          8288 kB          2762 kB                3               85
       system-uncached            28 kB             9 kB                3               86
       system-uncached          8288 kB          2762 kB                3               87
       system-uncached            28 kB             9 kB                3               88
       system-uncached          8288 kB          2762 kB                3               89
       system-uncached            28 kB             9 kB                3               90
       system-uncached          8288 kB          2762 kB                3               93
       system-uncached            28 kB             9 kB                3               94
         PROCESS TOTAL         49520 kB         19215 kB
----------------------
dmabuf total: 66568 kB kernel_rss: 384 kB userspace_rss: 157284 kB userspace_pss: 66183 kB
```

1.5.7 cpu_monitor

cpu_monitor 工具是动态的监测工具，除了检测当前 CPU 多核的消耗情况，还可以用于分析实时的内存增长情况。

使用说明：

```
cpu_monitor -m 1000
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
venus-a1:/ # cpu monitor -m 1000
-----------------------------------A50 --Mem State [unit:MB]-----------------------------------
Memory-Total:   982   Swap-Total:   737   Swap-Free:   680   Vma-Total:   240   Vma-Free:   240
Buffer:   1   KernStack:   30   Shmem:   3   Gpu:   0
Ion cma:   0   caverout:   0   sytem:   0   contig:   0
Total-used:  886   Total-freed:  28   Total-lost:  68
  Anon  Slab  Cache Sysfre Cmafre|  pgal[nor] pgfree pgfalt  fimap  kswap  dirty  wback   rbio   wbio
   217    43    532    27     0|       880   1745    967     12      2     14    304      0    104
   217    43    532    28     0|        88    201    127      0      0      1      5      0      0
   213    43    529    33     0|      1678   4893   7744     30      1    122     67      0    340
   213    43    529    32     0|       531    642   2619      0      0     -1     18      0      0
   216    43    529    28     0|      1702    733   6354     23      0     12      1    396     84
   217    43    530    26     0|      4928   4698   8729     24      2     92    601   1468    268
   217    43    530    27     0|      2894   3076   5961     16      0     13      1     40      4
   216    43    525    34     0|      4470   6358   9860   1615      2      0   1189      0      0
   216    43    526    31     0|      1818   1799   9478     97      0      0     37      0    160
   216    43    526    31     0|       925    733   3003      0      0     83     33    128    120
   218    43    526    28     0|      2584   2338  12495      1      0     36     66    204    268
   217    43    526    28     0|       239    430    160      9      0     27      1      0      4
   217    43    526    31     0|       258    520    271      2      0     22     36      0    140
   218    43    527    31     0|       503    690    747    264      0      2      0      4      0
   218    43    527    31     0|       209    156    140      0      0      7      0      0      0
   218    43    527    30     0|       728    721    546      0      0      6      1      0      4
```

图 1-1: cpu_monitor

如上图，红色方框粗略显示了当前系统的内存情况：

- Memory-Total 表示当前系统总的内存大小
- Swap-Total 表示当前系统总的 swap 大小
- Swap-Free 表示当前系统剩余的 swap 大小
- Vma-Total 表示当前系统总的虚拟地址空间，即 vmalloc 可分配的大小
- Vma-Free 表示当前系统剩余的虚拟地址空间大小，这里工具在 4.9 内核上支持有误。（不准确）
- Buffer
- KernStack 表示内核进程栈页面大小

红色方框下面一行的每一列的含义：

- Anon 表示系统匿名页
- Slab 表示内核 slab 大小
- Cache 表示系统 file 页面
- Sysfree 表示系统可用剩余内存
- Cmafree 表示系统可用 cma 大小
- Kswap 表示内核回收内存次数，此值若变大表示内存严重不足

1.6 内存相关的常用调试节点

1.6.1 proc/meminfo

系统内存总体使用情况，包含以下内容：

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

- MemTotal：总内存
- MemFree：完全空闲内存
- Buffers/Cached：缓存内存
- SwapTotal/SwapFree：交换空间

1.6.2 proc/vmallocinfo

vmalloc 分配区域信息，显示以下内容：

- 虚拟地址范围
- 分配大小
- 调用者信息

1.6.3 proc/buddyinfo

内存碎片诊断，显示以下内容：

- 每个 NUMA 节点
- 不同 order 的连续空闲页块数量

1.6.4 proc/pagetypeinfo

页面类型分布，包含以下内容：

- 不同迁移类型页面的统计
- 每个 zone 的页面分配情况

1.6.5 proc/zoneinfo

内存分区详细信息，显示以下内容：

- 每个内存分区的统计
- 保护水位线（low/high/min）
- 页面回收统计

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.6.6 proc/vmstat

虚拟内存统计计数器：

- pgscan_kswapd：kswapd 扫描页数
- pgsteal_kswapd：kswapd 回收页数
- oom_kill：OOM 触发次数

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 Android 上的内存管理机制

2.1 lmkd

lmkd（Low Memory Killer Daemon）是 Android 系统中的低内存终止守护进程，用来监控运行中 android 系统内存的状态，通过终止最不必要的进程来应对内存压力较高的问题，使系统以可接受的水平运行。

2.1.1 lmkd 的配置

lmkd 有很多的配置参数，但通常我们需要使用到的只有一部分

2.1.1.1 调整 ro.lmk.downgrade_pressure

原生 ro.lmk.downgrade_pressure 的默认值为 60，但这个值再低于 1GB 的平台上，会导致无法触发 lmkd，在低于 1GB 内存的平台上，这个值我们修改为 95，也就是说 mem_pressure 的值小于 95 时就会去杀进程，它的计算公式是：

```
mem_pressure = (mem_usage * 100) / memsw_usage
```

memory.usage_in_bytes 表示当前 memory 的使用；

memory.memsw.usage_in_bytes 表示当前 memory+swap 的使用；

2.1.1.2 调整 ro.lmk.upgrade_pressure

原生 ro.lmk.upgrade_pressure 的默认值为 40，这个值通常不需要修改

上述两个参数都在方案里修改，文件是/android/device/softwinner/venus-a1/venus_a1.mk

```
PRODUCT_PROPERTY_OVERRIDES += \
    ro.lmk.downgrade_pressure=95 \
    ro.lmk.upgrade_pressure=50

```

图 2-1: lmkd 配置

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.1.1.3 调整 ro.lmk.critical

这个值会决定不会北 lmkd 杀死进程的级别，级别的定义在 frameworks/base/services/core/java/com/android/server/am/ProcessList.java 文件中，常见的定义如下：

```
    public static final int HOME_APP_ADJ = 600;
    public static final int SERVICE_ADJ = 500;
    public static final int HEAVY_WEIGHT_APP_ADJ = 400;
    public static final int BACKUP_APP_ADJ = 300;
    public static final int PERCEPTIBLE_LOW_APP_ADJ = 250;
    public static final int PERCEPTIBLE_MEDIUM_APP_ADJ = 225;
    public static final int PERCEPTIBLE_APP_ADJ = 200;
    public static final int VISIBLE_APP_ADJ = 100;
    public static final int PERCEPTIBLE_RECENT_FOREGROUND_APP_ADJ = 50;
    public static final int FOREGROUND_APP_ADJ = 0;
```

非 GMS 方案希望前台的应用在内存极端的情况下也不会北 lmkd 杀死，可以配置 ro.lmk.critical=1，GMS 方案不能配置。

2.2 kswapd

2.2.1 内核的内存回收机制

内核的内存回收主要有两个，一个是 kswapd 进程对内存进行周期检查，以保证平常状态下剩余内存尽可能足够，另外一个是直接内存回收（direct page reclaim），就是当内存分配时没有空闲内存可以满足要求时，触发直接内存回收。

2.2.2 内存水位标记

Linux 内核使用水位标记 (watermark) 的概念来描述这个压力情况，linux 为内存的使用设置了三种内存水位标记，high，low，min，他们的含义分别为：

- 剩余内存在 high 以上表示内存剩余较多，目前内存使用压力不大
- [low~high] 的范围表示目前剩余内存存在一定压力
- [min~low] 表示内存开始有较大的压力，剩余内存不多了
- Min 是最小的水位标记，当剩余内存达到这个状态时，说明内存面临很大的压力
- 小于 min 这部分内存，内核是保留给特定情况使用，一般不会分配

2.2.3 触发 kswapd 的时机

当系统剩余内存低于 watermark[low] 的时候，内核的 kswap 开始起作用，进行内存回收，知道内存达到 watermark[high] 的时候停止。如果内存消耗导致剩余内存达到或者超过 watermark[min] 时，就会触发直接

版权所有 © 珠海全志科技股份有限公司。保留一切权利

内存回收 (driect reclaim)。

2.2.4 触发 kswapd 对系统的影响

当系统发生大量的 kswapd 时，表示内存严重不足了，虽然能够将一些不活跃的进程交换到 swap 分区，但是 kswapd 进程是占用 cpu 的，它很有可能会导致系统卡顿。所以选择合适的 kswap 触发阈值是很有必要的，并不是触发的越多越好。

2.3 ZRAM

zram 是 Linux 内核的一项功能，可提供虚拟内存压缩。zram 通过在 RAM 内的压缩块设备上分页，直到必须使用硬盘上的交换空间，以避免在磁盘上进行分页，从而提高性能。由于 zram 可以用内存替代硬盘为系统提供交换空间的功能，zram 可以在需要交换/分页时让 Linux 更好利用 RAM，在物理内存较少的旧电脑上尤其如此。

zram 在 2009 年的时候就进了 kernel 的 staging 目录，并于 2014 年 3 月 30 日发布的 3.14 版本正式合并入 Linux 内核主线。在 2014 年 6 月 8 日发布的 3.15 版本的 Linux 内核中，zram 已可支持 LZ4 压缩算法，而 LZO 仍然作为默认的压缩后端。内核 3.15 中的修改还改进了性能，以及经由 sysfs 切换压缩算法的能力。

zram swap 主要原理就是从内存分配一块区域出来用作 swap 分区，每次如果内存空间不够了，不是把应用程序杀掉，而是把应用程序所占用的内存数据复制到 swap 分区，等切换回来的时候就可以直接把这部分数据恢复到内存当中，节省重新开启所需的时间。而被放到 swap 分区的应用程序，所占用的内存都是被压缩过的，比如，微信在普通内存中占用 50 MB 的空间，如果压缩率为 0.4，则放到 swap 分区里面的数据只需要 20 MB 的空间，这样 swap 分区里面就可以存放更多后台临时不用的应用程序，变相扩展了内存的大小。

2.3.1 打开 zram 的配置

```
CONFIG_ZRAM=y
CONFIG_SWAP=y
CONFIG_LZ4_COMPRESS=y
CONFIG_LZ4_DECOMPRESS=y
CONFIG_DECOMPRESS_LZ4=y
CONFIG_HAVE_KERNEL_LZ4=y
CONFIG_RD_LZ4=y
CONFIG_CRYPTO_LZ4=y
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.3.2 设置 zram 的参数

2.3.2.1 zram 的大小

配置 zram 大小的文件是在 device/softwinner/venus-a1/fstab.sun8iw15p1，在 Android N 以前的方案 zramsize 一般都是配置为 256MB 或者 128MB 的，自从 Android O 以后，对于 1G 内存设备，Google 推荐为了性能更佳推荐 zramsize 配置为整个系统内存的 75%。

```
dev/block/zram0    none    swap    defaults    zramsize=75%
```

2.3.2.2 zram 的压缩算法

查看当前支持的压缩算法，A50 使用的是 lz4 压缩算法

```
venus-a1:/sys/block/zram0 # cat comp_algorithm
lzo [lz4] deflate
```

图 2-2: lz4

2.3.2.3 zram 的最大压缩流

查看当前支持的最大压缩流，A50 使用默认的最大压缩流为 1

```
venus-a1:/sys/block/zram0 # cat max_comp_streams
1
```

图 2-3: zram 最大压缩流

可以在 init rc 文件通过 write 命令来修改上述的压缩算法以及最大压缩流，命令举例：

```
write /sys/block/zram0/comp_algorithm lz4
write /sys/block/zram0/max_comp_streams 4
```

2.3.3 ZRAM 回写配置

不管是否是低内存设备, 都会开启 ZRAM 的支持, 对于低内存设备, 还将开启内存扩展 (ZRAM 回写) 功能, ZRAM 回写的配置如下:

文件路径: android/device/softwinner/saturn/common/system/config.mk

```
#zram write back
PRODUCT_PROPERTY_OVERRIDES += \
    persist.sys.zram_enabled = 1 \
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    ro.zram.first_wb_delay_mins = 180 \
    ro.zram.mark_idle_delay_mins = 60 \
    ro.zram.periodic_wb_delay_hours = 24 \
```

系统开机的时候, 会识别出 dram 的大小, 以属性 ro.boot.dramsize 的方式传给 Android, 以供自动配置, 每次开机的时候, 会执行以下脚本:

文件位置: android/device/softwinner/common/init.ram.rc

这个文件会根据实际内存的大小进行配置, 对于总 DRAM 大小小于 1GB 的设备, 会默认开启 256M 的内存扩展 (ZRAM 回写), 对于超过 1GB 的设备, 则需要

通过属性 persist.sys.ramconfig 配置内存扩展的大小:

这个值的配置只是以下值:

gsi: 不开启内存扩展 (GSI 固件使用)

none: 不开启内存扩展

512: 开启 512M 内存扩展

1024: 开启 1024M 内存扩展

2048: 开启 2048M 内存扩展

4096: 开启 4096M 内存扩展

注意: 这个值必须被配置, 否则将导致启动的时候没有挂载 ZRAM

如果烧写了 GSI 固件, GSI 固件下, persist.sys.ramconfig 会被自动设置为 gsi

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 常用的内存优化方法

3.1 设置为 low device

Google 从 Android O 开始，认为设备内存低于或者等于 1G 的设备认为是低内存设备，特定义为 Android Go 设备，会使用 Go 的应用，launcher 等，显示渲染方面会为 go 设备而进行相应的内存优化。

只需要设置一个属性即可。

```
ro.config.low_ram=true
```

3.2 优化预留内存

预留内存是系统启动时，将这部分内存设置为 memoryblock_reserve，其他模块无法使用，只允许被设定的模块使用。

```
00007: /* kernel used */
00008: /memreserve/ 0x40020000 0x00000800; /* super standby rang:      [0x40020000~0x41020800], size = 2K  */
00009: /memreserve/ 0x48000000 0x01000000; /* atf:                     [0x48000000~0x49000000], size = 16M */
00010:
00011: /* tf used */
00012: /memreserve/ 0x48100000 0x00004000; /* arisc dram code space range:  [0x48100000~0x48104000], size = 16K */
00013: /memreserve/ 0x48104000 0x00001000; /* arisc para cfg range:         [0x48104000~0x48105000], size = 4K  */
00014: /memreserve/ 0x48105000 0x00001000; /* arisc message pool range:     [0x48105000~0x48106000], size = 4K  */
00015: /* ramoops used */
00016: /memreserve/ 0x48106000 0x00060000; /* ramoops range :          [0x48106000~0x48166000], size = 384K */
```

图 3-1: memoryblock_reserve

上面的 dts 需要定义的预留内存，除了这里有定义，在 sys_config.fex 也有定义 DRM 的预留内存。A50 只在 widevine 为 L1 时才使能下面的配置，L3 默认关闭。

```
1405 ;-----------------------------------------------------------------------------------
1406 ;dram_region_mbytes  : Widevine L1 set 64Mbyte,Widevine L3 set 0Mbyte to save ram
1407 ;-----------------------------------------------------------------------------------
1408 ;[secure]
1409 ;dram_region_mbytes        = 64
1410 ;-----------------------------------------------------------------------------------
```

图 3-2: memoryblock_reserve2

3.2.1 如何查看详细的预留内存

在 env.cfg 里往 cmdline 添加一项：memblock=debug，可以将预留内存的详细情况打印出来。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

下面是某方案预留内存的分布情况。

```
[    0.000000] memblock_reserve: [0x00000040100000-0x000000410b944b] flags 0x0
    arm_memblock_init+0x30/0x184,size = 16101 KB
[    0.000000] memblock_reserve: [0x00000042000000-0x000000421a0ab9] flags 0x0
    arm_memblock_init+0xfc/0x184,size = 1666 KB
[    0.000000] memblock_reserve: [0x00000040004000-0x00000040007fff] flags 0x0
    arm_mm_memblock_reserve+0x1c/0x24,size = 16 KB
[    0.000000] memblock_reserve: [0x00000044000000-0x000000440147df] flags 0x0
    early_init_dt_reserve_memory_arch+0x20/0x24,size = 81 KB
[    0.000000] memblock_reserve: [0x00000040020000-0x000000400207ff] flags 0x0
    early_init_dt_reserve_memory_arch+0x20/0x24,size = 2 KB
[    0.000000] memblock_reserve: [0x00000048000000-0x00000048ffffff] flags 0x0
    early_init_dt_reserve_memory_arch+0x20/0x24,size = 16384 KB //16MB
[    0.000000] memblock_reserve: [0x00000048100000-0x00000048103fff] flags 0x0
    early_init_dt_reserve_memory_arch+0x20/0x24,size = 16 KB
[    0.000000] memblock_reserve: [0x00000048104000-0x00000048104fff] flags 0x0
    early_init_dt_reserve_memory_arch+0x20/0x24,size = 4 KB
[    0.000000] memblock_reserve: [0x00000048105000-0x00000048105fff] flags 0x0
    early_init_dt_reserve_memory_arch+0x20/0x24,size = 4 KB
[    0.000000] memblock_reserve: [0x00000048106000-0x00000048165fff] flags 0x0
    early_init_dt_reserve_memory_arch+0x20/0x24,size = 384 KB
[    0.000000] memblock_reserve: [0x0000007c000000-0x0000007fffffff] flags 0x0
    early_init_dt_reserve_memory_arch+0x20/0x24,size = 65536 KB //64MB
[    0.000000] memblock_reserve: [0x00000042000000-0x000000421a0ab9] flags 0x0
    early_init_dt_reserve_memory_arch+0x20/0x24,size = 1666 KB
[    0.000000] memblock_reserve: [0x0000007a000000-0x0000007bffffff] flags 0x0
    memblock_alloc_range_nid+0x40/0x58,size = 32768 KB //32MB
[    0.000000] cma: Reserved 32 MiB at 0x7a000000
//下面是对上面的汇总
[    0.000000] MEMBLOCK configuration:
[    0.000000]  memory size = 0x40000000 reserved size = 0x8172ee6  //预留内存总大小129MB
[    0.000000]  memory.cnt  = 0x1
[    0.000000]  memory[0x0]     [0x00000040000000-0x0000007fffffff], 0x40000000 bytes flags:
    0x0
[    0.000000]  reserved.cnt  = 0x7
[    0.000000]  reserved[0x0]   [0x00000040004000-0x00000040007fff], 0x4000 bytes flags: 0x0
[    0.000000]  reserved[0x1]   [0x00000040020000-0x000000400207ff], 0x800 bytes flags: 0x0
[    0.000000]  reserved[0x2]   [0x00000040100000-0x000000410b944b], 0xfb944c bytes flags: 0
    x0  //内核代码段，16MB
[    0.000000]  reserved[0x3]   [0x00000042000000-0x000000421a0ab9], 0x1a0aba bytes flags: 0
    x0
[    0.000000]  reserved[0x4]   [0x00000044000000-0x000000440147df], 0x147e0 bytes flags: 0x0
[    0.000000]  reserved[0x5]   [0x00000048000000-0x00000048ffffff], 0x1000000 bytes flags: 0
    x0  //ATF占用16MB
[    0.000000]  reserved[0x6]   [0x0000007a000000-0x0000007fffffff], 0x6000000 bytes flags: 0
    x0  //cma+widevide = 96MB

//从下面来看，预留总大为139MB，与前面的129M相差10MB，应该是还有10MB是给内核镜像用。
[    0.000000] Memory: 905492K/1048576K available (9216K kernel code, 1378K rwdata, 2928K
    rodata, 1024K init, 381K bss, 110316K reserved, 32768K cma-reserved, 163840K highmem)
```

其中 ATF 预留的 16MB 内存，分布给 atf、cpus、共享内存、optee 用，它们的内存分布如下：

ATF	CPUS	共享内存	OPTEE
1MB	1MB	4MB	10MB

版权所有 © 珠海全志科技股份有限公司。保留一切权利

不管是安全还是非安全方案，ATF 都是预留了 16MB，针对很多平台其实是用不到的，尤其是一些根本没有安全需求的平台，那么可以考虑将 ATF 部分缩减为 2MB，只需要预留 2MB 给 cpus 使用就足够了，与此同时，需要把 TEE 从内核模块里面去掉。

除了 ATF 内存，DRM 的内存也是一个优化项目，对于非 Widevine L1 认证的项目，

3.3 优化 GC 回收机制

对于 LowRam 设备可能会有很多 GC，Google 推荐通过修改参数可以减少 GC，从而提升内存使用率。

下面的修改可以在方案的.mk 文件里修改。

```
# Reduces GC frequency of foreground apps by 50%
PRODUCT_PROPERTY_OVERRIDES += dalvik.vm.foreground-heap-growth-multiplier=2.0
```

3.4 优化 awbms

awbms 服务是全志独有的后台管理服务，由于它采用的是 app_process 方式启动，它会复制 zygote 的一份内存，导致 awbms 占用 app_process 一份内存，这不是必要的，可以采用在 SystemServer 里面去静态启动 awbms 服务，这样不会有复制的内存，可以减少 10MB 左右。如果某些平台不需要 awbms，则可以在 common.mk 里面直接去掉它，不把它编译进 Android 即可。

3.5 优化 SystemUI

当系统启动后，首先查看 systemui 对应的 PID，然后再使用 dumpsys meminfo PID，可以发现 systemui 对应的 Native Heap 比较大，说明它可以释放的 GC 空间比较多。为了优化 systemui 的内存占用，我们在 awbms 里面对 systemui 进行手动 GC 操作，原理是等系统启动后，10 分钟后发送 kill -10 $PID 命令。大约能够节省 10~20MB 左右。

3.5.1 裁剪法优化 SystemUi

某些平台，不需要使用状态栏、锁屏界面等功能，那么可以考虑将其从 systemui 中移除，解决办法是在方案目录进行 overlay 修改。

```
19    <!-- Empty stub status/navigation bar -->
20    <!-- com.android.systemui.statusbar.tv.TvStatusBar or com.android.systemui.statusbar.phone.StatusBar -->
21    <string name="config_statusBarComponent" translatable="false">com.android.systemui.statusbar.tv.TvStatusBar</string>
22
23    <!-- Disable KeyguardSerivce -->
24    <bool name="config_enableKeyguardService">false</bool>
```

图 3-3: systemui 优化

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.6 通过虚拟机参数优化 SystemServer

Android O 以后，Google 为减少内存占用，特意在虚拟机编译阶段引入了 speed-profile 参数，只需要配置上这个参数就能够优化 System 进程的内存占用。

```
PRODUCT_SYSTEM_SERVER_COMPILER_FILTER := speed-profile
```

3.7 gms 包的裁剪

某些平台产品可能不需要 gms 包或者只需要某几个应用，可以对 gms 包进行删除或者裁剪，从而减少对系统内存的占用。

3.8 优化 vmalloc

通过 cat /proc/vmalloc 命令可以查看具体哪些模块分配了多少虚拟内存。在开发中发现有 2 个模块的虚拟内存可以进一步优化，一个是 iotable_init，一个是 pcm_buffer。在内存开发中，可以重点关注一下这两个模块的虚拟内存占用情况。

3.9 优化 slab

Slab 分可回收与不可回收，一般来说 32bit 的平台，slab 总大小应该为 50MB 以下，64bit 的平台 slab 总大小应该为 100MB 以下，如果超出这个大小说明需要排查一下什么原因。

CONFIG_SLUB_DEBUG 在量产的 SDK 里需要去掉，否则将增大 slab 的大小，意味着整个 kernel 的内存占用将增大。

3.10 裁剪不必要的应用

使用 dumpsys meminfo 可以查看 Android 哪些进程占用的内存多，针对项目开发过程中，可以查看是否有应用是不必要的，可以根据实际情况来裁剪，以免对系统内存造成浪费。

3.11 关掉蓝牙和自动旋转

某些平台不要蓝牙以及旋转功能，例如 OTT 平台，那么可以考虑将其设置为 flase，而实现内存的优化。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
overlay/frameworks/base/packages/SettingsProvider/res/values/defaults.xml
<bool name="def_bluetooth_on">false</bool>
<bool name="def_accelerometer_rotation">false</bool>
```

3.12 显示优化

通过 dumpsys SurfaceFlinger 来查看显示消耗的内存，根据实际情况来判断是否有浪费内存的情况。是否有优化空间，例如针对显示分辨率要求不高的产品可以选择 RGB565 的方式显示。

```
Allocated buffers:
0xa81255b0: 3600.00 KiB | 1280 (1280) x  720 |    1 |        1 | 0x1a00 | FramebufferSurface
0xa8125620: 3600.00 KiB | 1280 (1280) x  720 |    1 |        1 | 0x1a00 | FramebufferSurface
0xa8125690: 3600.00 KiB | 1280 (1280) x  720 |    1 |        1 | 0x1a00 | FramebufferSurface
0xa81273e0: 8100.00 KiB | 1920 (1920) x 1080 |    1 |        1 | 0xb02 | com.android.systemui.ImageWallpaper#0
0xa81274c0: 3600.00 KiB | 1280 (1280) x  720 |    1 |        1 | 0xb02 | com.android.tv.launcher/com.android.tv.launcher.MainSettings#0
0xa8127760: 3600.00 KiB | 1280 (1280) x  720 |    1 |        1 | 0xb02 | com.android.tv.launcher/com.android.tv.launcher.MainSettings#0
0xa81277d0: 3600.00 KiB | 1280 (1280) x  720 |    1 |        1 | 0xb02 | com.android.tv.launcher/com.android.tv.launcher.MainSettings#0
Total allocated (estimate): 29700.00 KB
```

图 3-4: SurfaceFlinger

另外，针对内核 disp 驱动排查是否有预留内存给 logo 使用，这个需要在预留内存里面分析内存使用情况。

3.13 合理配置 cma 的预留大小

cma 的预留大小影响着整个系统的内存，如果 cma 预留的太大，则会导致系统内存的浪费，如果预留的太小，会导致某些场景无法获取足够的 cma 而导致系统内存异常。所以需要根据实际的产品需求来定制 cma 的预留大小，一般而言，1G 的设备，cma 预留内存大小不要超过 320MB。

另外，针对有 iommu 硬件的平台，则基本不需要预留 cma，因为 iommu 就是为了解决 cma 的提前预留的弊端。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。